Q15: Context Windows, Retrieval, and Why Context Is Not Memory

DLS 2026 — module author: Kukla

October 2026

Learning objectives

- Explain what a context window is and its cost structure.
- Describe retrieval-augmented generation (RAG) and when it beats a longer context.
- Articulate why context is not memory, and how agent memory systems fill the gap.

Companion videos (watch in order):

- *How context windows work* — Matt Pocock (9:33). https://www.youtube.com/watch?v=-uW5-TaVXu4
- *Agent Short/Long-term Memory vs RAG Explained* — Cloud With Raj (14:43). https://www.youtube.com/watch?v=O8WT92hk3tw

1 The question

Student responses #13 and #15 ask how context windows, retrieval, and conversation memory work, and how to avoid hallucination without deleting context. The through-line: a model's *context window* is a powerful but finite and expensive working memory, retrieval extends what it can ground on, and neither is the same as learning—a distinction with direct practical consequences.

2 The context window and its cost

The context window is the maximum number of tokens the model attends over in one forward pass. Everything the model "knows right now"—the system prompt, the conversation, retrieved documents—must fit inside it. The catch is cost: self-attention is $O(n^2)$ in sequence length n for compute, and the **KV cache** (stored keys/values for every token, so generation need not recompute the past) grows *linearly* in n and is often the real memory bottleneck at long context. Doubling context does not double cost—it can quadruple attention compute and double KV memory—so "just make the window bigger" is not free. Techniques like grouped-query attention, sliding-window/sparse attention, and paged KV caches exist precisely to tame this (serving module territory).

3 Retrieval-augmented generation

Rather than stuff everything into context, **RAG** retrieves only the relevant pieces at query time [1]. The pipeline: embed documents into vectors, store them in a vector index, embed the query, fetch the top-k nearest chunks, and place them in the context as grounding. Benefits over a huge window:

you can draw on a corpus far larger than any window, you pay attention cost only on what you retrieved, and—critically for #15—the model can *cite* grounded sources, which reduces hallucination because the answer is conditioned on supplied evidence rather than parametric guesswork. RAG's failure modes are its own: bad retrieval (missing or irrelevant chunks) caps answer quality, and the model can still ignore or misread what was retrieved.

4 Long context vs. retrieval

These are complementary, not rivals. Long context wins when the relevant information is a coherent whole that must be reasoned over jointly (a long contract, a codebase file). Retrieval wins when the knowledge is vast, sparse, frequently updated, or must be attributed. A common pattern combines them: retrieve to narrow the corpus, then use a long window to reason over the retrieved set. Deciding between them is an engineering judgment about information density, freshness, and cost.

5 The anatomy of a RAG query

Walking one query through end to end makes the moving parts concrete. (1) **Indexing (offline)**: each document is split into chunks, each chunk is passed through an embedding model to produce a vector, and the vectors are stored in a similarity index. (2) **Query time**: the user's question is embedded with the *same* model, the index returns the top-k nearest chunks by cosine similarity, and those chunks are concatenated into the prompt ahead of the question. (3) **Generation**: the model answers conditioned on the retrieved evidence, ideally citing it. The companion code implements exactly this loop with a toy embedding. Each stage has a failure point students should know: chunking too coarsely buries the answer, too finely loses context; an embedding model mismatched to the domain retrieves the wrong chunks; and k too small misses evidence while k too large wastes context and can distract the model. Retrieval quality is usually the dominant lever on end-to-end RAG quality—more than the generator.

6 Hallucination and grounding

The students' specific worry—avoiding hallucination without deleting context—maps onto this pipeline. Grounding helps because an answer conditioned on supplied, relevant text is less likely to be fabricated than one drawn purely from parametric memory. But grounding is not a guarantee: the model can still misread a retrieved passage, blend it with a wrong prior, or answer confidently when retrieval returned nothing useful. Robust systems therefore add checks—verify that cited spans actually support the claim, abstain when retrieval confidence is low, and surface sources so a human can audit. "Don't delete context" becomes "keep the *right* context available and prove the answer rests on it."

7 Why context is not memory

Here is the crux of the students' worry about "avoiding hallucination without deleting context." The context window is **volatile working memory**: when the conversation ends, or when old tokens scroll out of the window, that information is gone—it is never written back into the weights (module Q14's long-term store). So a chatbot does not "remember" you across sessions; it re-reads whatever is re-supplied or retrieved. **Agent memory systems** engineer around this: they persist facts to

an external store (a database or vector index), summarize or compress old turns, and selectively *re-inject* the relevant pieces into context on later turns. That is how an agent appears to have long-term memory without retraining—and it is why "avoid deleting context" is really a question of *what to persist, summarize, and retrieve*, not of making the window infinite. The companion code (`code/tiny_rag.py`) implements a minimal embed–index–retrieve loop to make this concrete.

8 Summary

A context window is finite, $O(n^2)$-expensive working memory with a linearly growing KV cache. RAG retrieves only relevant chunks, scales beyond any window, and grounds answers to cut hallucination, at the cost of retrieval quality. Long context and retrieval are complementary. Crucially, context is volatile working memory, not learning; agent memory systems persist, compress, and re-inject information to simulate long-term memory without touching the weights.

References

[1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *NeurIPS*, 2020.